

ROUTING

ANGULAR ROUTER

- Angular ist eine SPA, benötigt also eigene Logik um Navigieren zu können
- Kern: Angular Router
 - Service der Routing Aufgaben übernimmt
 - Konfiguration via Route Objekten

ROUTE

- Entspricht einem Pfad auf eine Ansicht
- Kann beliebig verschachtelt werden
- Ermöglicht sogenanntes "Lazy Loading"

- Bestandteile
 - path
 - Pfad zu einer Component / anderen Route
 - component
 - Die darzustellende Component
 - canActivate
 - Array an Guards, welche bestimmen ob eine Route angesteuert werden kann
 - canDeactivate
 - Array an Guards, welche bestimmen ob eine Route verlassen werden kann
 - resolve
 - Objekt welches diverse Resolver entgegen nimmt (Routing ist angehalten bis die Daten da sind)

- Beispiel Routen

```
const routes = {  
  path: 'parent-route',  
  children: [  
    {  
      path: '',  
      redirectTo: 'child-route-0',  
      pathMatch: 'full',  
    },  
    {  
      path: 'child-route-0',  
      component: Child0Component  
    },  
    {  
      path: '**',  
      component: PageNotFoundComponent  
    }  
  ]  
}
```

GUARD

- "Bewachen" die Route auf der sie liegen
- Können ein Umleiten auf andere Route erzwingen
- Reihenfolge der Guards ist wichtig

- Beispiel:

```
export const isAuthenticatedGuard = () => {  
  const authService = inject(AuthService);  
  
  return authService.isAuthenticated();  
}
```

- Beispiel (mit Navigation):

```
export const isAuthenticatedGuard = () => {  
  const authService = inject(AuthService);  
  const router = inject(Router);  
  
  if (!authService.isAuthenticated()) {  
    router.navigate(['login']);  
    return false;  
  }  
  
  return true;  
}
```

RESOLVER

- Ermöglicht das Laden von Daten von Daten sobald eine Route angesteuert wird
- Router wartet mit Auflösen der Route bis Daten vorhanden sind

- Beispiel:

```
export const productResolver: ResolveFn<Product> = (
  route: ActivatedRouteSnapshot,
  state: RouterStateSnapshot,
  productService = inject(ProductService)
) => {
  return productService.getActiveProduct();
};
```

LAZY LOADING

- Komponenten werden direkt mit Routen verdrahtet
- Problem: "eager loading"
 - Initiale Bundles bzw. Anzahl an zu ladenden Fragmenten wird größer
- Lösung: "lazy loading"
 - Komponenten inkl. Abhängigkeiten werden erst beim Ansteuern der Route geladen

- Beispiel:

```
// routes-a.ts
export const routesA: Routes = [
  {
    path: 'route-a', component: ComponentA
  },
  {
    path: 'route-ab',
    loadChildren: () =>
      import('path-to-routes-b.ts').then(mod => mod.routesB),
  }
]

// routes-b.ts
export const routesB: Routes = [
  {path: 'route-b', component: ComponentB}
]
```

- Beispiel für einzelne Komponente:

```
export const routes: Routes = [
  {
    path: 'route-a', component: ComponentA
  },
  {
    path: 'route-b',
    loadChildren: () =>
      import('path-to-component-b.ts').then(mod => mod.ComponentB),
  }
]
```

KOMPONENTEN INPUT BINDING

KOMPONENTEN INPUT BINDING

- Es ist möglich den Router so zu konfigurieren, dass er Daten direkt in die Inputs von Komponenten mappen kann
- Dazu zählen
 - Routen Parameter
 - Query Parameter
 - Statische Routen Daten
 - Daten von Resolvieren
- Dazu übergeben wir der provideRouter-Funktion an zweiter Stelle withComponentInputBinding()

```
bootstrapApplication(App,
{
  providers: [
    provideRouter(appRoutes, withComponentInputBinding())
  ]
});
```

KOMONENTEN INPUT BINDING

```
{ path: 'new/:id', component: ProductForm }
```

- Ohne Komponenten Input Binding:

```
export class ProductForm {  
  id = -1;  
  constructor() {  
    inject(ActivatedRoute).paramMap.subscribe((paramMap) => {  
      const idParam = paramMap.get('id');  
      if (idParam) {  
        this.id = Number(idParam);  
      }  
    });  
  }  
}
```

- Mit Komponenten Input Binding:

```
export class ProductForm {  
  id = input(-1, { transform: numberAttribute });  
}
```

- Resolver ohne withComponentInputBinding:

```
export const productResolver: ResolveFn<Product> = (  
  route: ActivatedRouteSnapshot,  
  state: RouterStateSnapshot  
) => {  
  return inject(ProductData).getActiveProduct();  
};
```

```
export const routes: Routes = [  
  { path: 'product', component: ProductDisplay, resolve: { product: productResolver } }  
];
```

```
export class ProductDisplay {  
  routeData = toSignal(inject(ActivatedRoute).data);  
  product = this.routeData()[ 'product' ] as Product;  
}
```

- Resolver mit withComponentInputBinding:

```
export const productResolver: ResolveFn<Product> = (  
  route: ActivatedRouteSnapshot,  
  state: RouterStateSnapshot  
) => {  
  return inject(ProductData).getActiveProduct();  
};
```

```
export const routes: Routes = [  
  { path: 'product', component: ProductDisplay, resolve: { product: productResolver } }  
];
```

```
export class ProductDisplay {  
  product = input.required<Product>();  
}
```